# PROGRAMACIÓN VISUAL STUDIO MENU

PRÁCTICAS PARA ASISTENCIA

**Presenta:**
CHAVIRA RUIZ JAZMIN

**Materia:**

PROGRAMACIÓN

**Universidad:**

TECNOLÓGICO NACIONAL DE MÉXICO

La base de datos se utiliza para almacenar la información de los usuarios del sistema, como el nombre de usuario y la contraseña.
Permite registrar nuevos usuarios y verificar sus datos al momento de iniciar sesión, asegurando el acceso al sistema.

```sql
-- ==========================================================
-- SCRIPT SQL: App Login C# WinForms + MySQL + Patrón MVC
-- Autor: Tutorial Práctico | MySQL 8.x
-- ==========================================================
-- 1. Crear y seleccionar la base de datos
DROP DATABASE IF EXISTS app_login_csharp;
CREATE DATABASE app_login_csharp
 DEFAULT CHARACTER SET utf8mb4
 DEFAULT COLLATE utf8mb4_unicode_ci;
USE app_login_csharp;
-- 2. Tabla principal de usuarios
CREATE TABLE usuarios (
 id INT NOT NULL AUTO_INCREMENT,
 nombre VARCHAR(100) NOT NULL,
 apellido VARCHAR(100) NOT NULL,
 email VARCHAR(150) NOT NULL UNIQUE,
 usuario VARCHAR(60) NOT NULL UNIQUE,
 contrasena VARCHAR(255) NOT NULL,
 foto MEDIUMBLOB DEFAULT NULL, -- Hasta 16 MB
 nivel_seg TINYINT DEFAULT 0, -- 0=Muy débil ... 4=Muy fuerte
 intentos TINYINT DEFAULT 0, -- Contador intentos fallidos
 bloqueado TINYINT(1) DEFAULT 0, -- 1 = cuenta bloqueada
 ultimo_acc DATETIME DEFAULT NULL,
 fecha_reg DATETIME DEFAULT CURRENT_TIMESTAMP,
```

e 13 | SQL File 5* | SQL File 6* | SQL File 7* | SQL File 8* | SQL File 9* | SQL Additio

Limit to 1000 rows

```sql
  activo TINYINT(1) DEFAULT 1,
  PRIMARY KEY (id),
  INDEX idx_usuario (usuario),
  INDEX idx_email (email)
);
-- 3. Tabla de log de accesos
CREATE TABLE log_accesos (
  id INT NOT NULL AUTO_INCREMENT,
  usuario_id INT NOT NULL,
  tipo ENUM('LOGIN_OK','LOGIN_FAIL','LOGOUT') NOT NULL,
  ip_cliente VARCHAR(45) DEFAULT NULL,
  descripcion VARCHAR(200) DEFAULT NULL,
  fecha DATETIME DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (id),
  FOREIGN KEY (usuario_id) REFERENCES usuarios(id) ON DELETE CASC
  INDEX idx_fecha (fecha)
);
-- 4. Tabla de sesiones activas
CREATE TABLE sesiones (
  id INT NOT NULL AUTO_INCREMENT,
  usuario_id INT NOT NULL,
  token VARCHAR(100) NOT NULL UNIQUE,
  inicio DATETIME DEFAULT CURRENT_TIMESTAMP,
  expira DATETIME NOT NULL,
```

Autor
disable
manu
curren
tog

Context He

Output

e 13 | SQL File 5* | SQL File 6* | SQL File 7* | SQL File 8* | SQL File 9* | SQLAddi

Limit to 1000 rows

```
SET p_resultado = 'USUARIO_NO_EXISTE';
ELSEIF v_activo = 0 THEN
SET p_resultado = 'CUENTA_INACTIVA';
ELSEIF v_bloqueado = 1 THEN
SET p_resultado = 'CUENTA_BLOQUEADA';
ELSEIF v_pass = p_contrasena THEN
-- Credenciales correctas
UPDATE usuarios SET intentos = 0, ultimo_acc = NOW()
WHERE id = v_id;
INSERT INTO log_accesos (usuario_id, tipo, descripcion)
VALUES (v_id, 'LOGIN_OK', 'Acceso exitoso');
SET p_resultado = 'OK';
ELSE
-- Contraseña incorrecta
UPDATE usuarios SET intentos = intentos + 1 WHERE id = v_id;
INSERT INTO log_accesos (usuario_id, tipo, descripcion)
VALUES (v_id, 'LOGIN_FAIL', CONCAT('Intento ', v_intentos+1));
IF v_intentos + 1 >= 5 THEN
UPDATE usuarios SET bloqueado = 1 WHERE id = v_id;
SET p_resultado = 'CUENTA_BLOQUEADA';
ELSE
SET p_resultado = 'CREDENCIALES_INVALIDAS';
END IF;
END IF;
```

Auto
disab
man
curre
tog

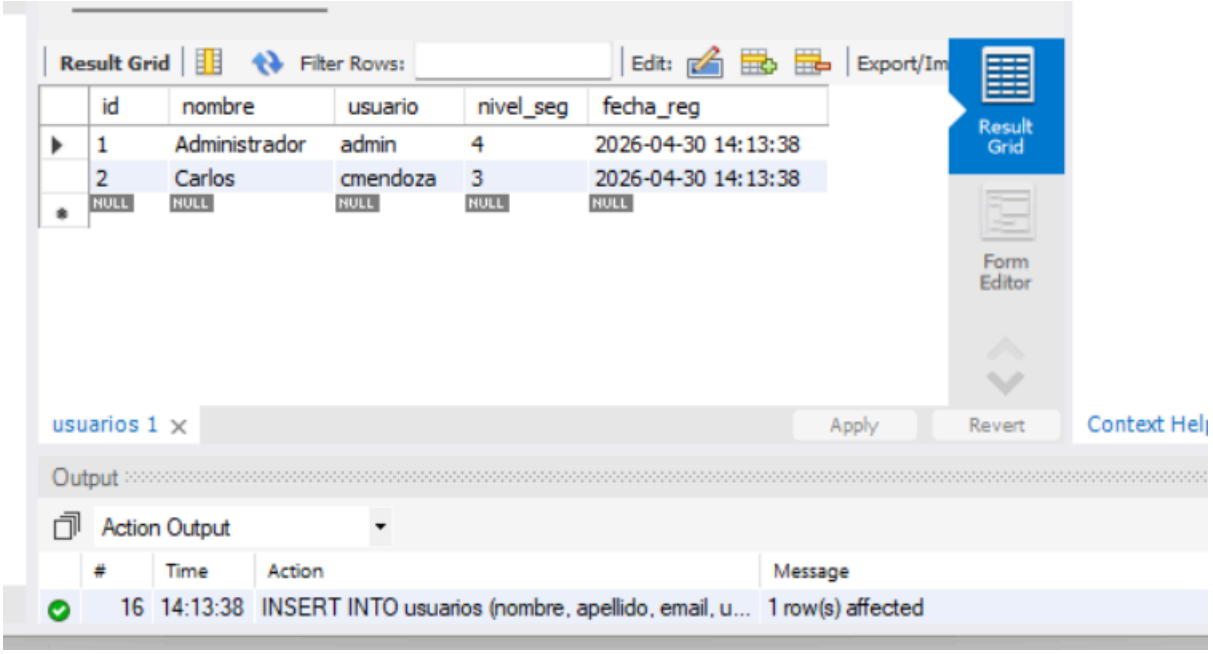
Context H

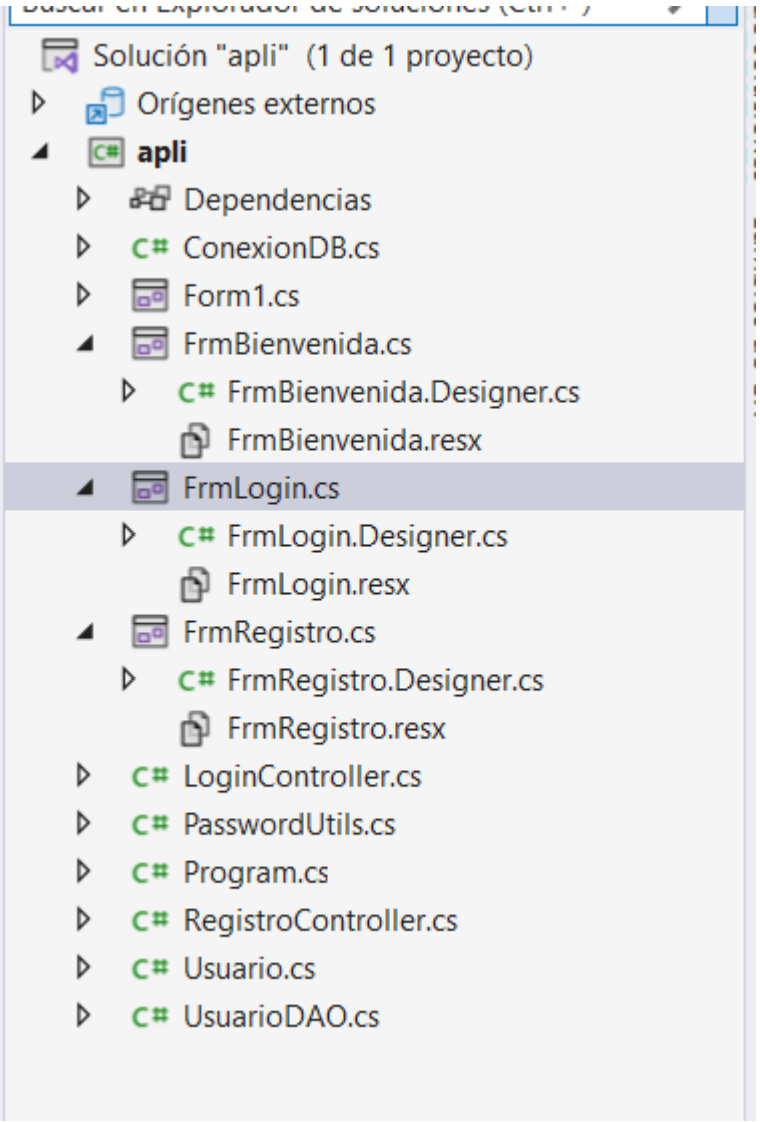
Output

```
END$$
DELIMITER ;
-- ═══════════════════════════════════════════════════════════
-- 6. Vista para reportes (sin datos sensibles)
-- ═══════════════════════════════════════════════════════════
CREATE VIEW v_usuarios_resumen AS
SELECT id, nombre, apellido, email, usuario,
 nivel_seg, intentos, bloqueado,
 ultimo_acc, fecha_reg, activo
 FROM usuarios;
-- ═══════════════════════════════════════════════════════════
-- 7. Datos de prueba
-- ═══════════════════════════════════════════════════════════
-- Contraseña Admin@2024 cumple los 5 requisitos de seguridad
INSERT INTO usuarios (nombre, apellido, email, usuario, contrase
VALUES ('Administrador', 'Sistema', 'admin@sistema.com', 'admin
4);
-- Contraseña: Test#9876
INSERT INTO usuarios (nombre, apellido, email, usuario, contrase
VALUES ('Carlos', 'Mendoza', 'carlos@demo.com', 'cmendoza', 'Tes
-- Verificar
SELECT id, nombre, usuario, nivel_seg, fecha_reg FROM usuarios;
-- ═══════════════════════════════════════════════════════════
-- Script completado: base de datos app_login_csharp lista
```

Automati
disabled. U
manually
current ca
toggle a

Context Help


Result Grid
Filter Rows:
Edit:
Export/Im
id
nombre
usuario
nivel_seg
fecha_reg
1
Administrador
admin
4
2026-04-30 14:13:38
2
Carlos
cmendoza
3
2026-04-30 14:13:38
NULL
Result Grid
Form Editor
usuarios 1
Apply
Revert
Context Hel
Output
Action Output
#
Time
Action
Message
16
14:13:38
INSERT INTO usuarios (nombre, apellido, email, u...
1 row(s) affected




La aplicación está desarrollada en C# con Windows Forms (.NET 8) y utiliza una base de datos MySQL para almacenar la información de los usuarios.

El sistema sigue una estructura basada en el patrón MVC (Modelo – Vista – Controlador), que permite separar responsabilidades y organizar mejor el código.
¿Cómo está organizado?

El proyecto se divide en 3 partes principales:
1. Modelo (Modelo)

Se encarga de:

Manejar la base de datos
Definir las clases de datos (usuarios)
Validar información (como contraseñas)
Vista (Formularios)

Son las pantallas que ve el usuario:
Login
Registro
Bienvenida
Aquí solo hay interfaz, no lógica compleja.
3. Controlador

Es el intermediario entre la vista y el modelo:

Recibe datos del usuario
Procesa la lógica
Llama a la base de datos
Flujo general del sistema
El usuario abre la aplicación
Se muestra el formulario de Login
El usuario ingresa datos
El controlador valida la información
Se consulta la base de datos
Si es correcto → se abre la pantalla de bienvenida
Si no → se muestra un error

BIENVENIDA
Ubicación: Vista

Descripción:
Pantalla que se muestra después de un login exitoso.

Responsabilidad:

Mostrar información del usuario

Métodos principales:

Constructor que recibe un objeto Usuario

Importancia:
Confirma el acceso exitoso al sistema.

```
using apli.Modelo;

namespace apli
{
    3 referencias
    public partial class FrmBienvenida : Form
    {
        1 referencia
        public FrmBienvenida(Usuario u)
        {
            InitializeComponent();
            lblTexto.Text = $"Bienvenido {u.NombreCompleto}";
        }

        1 referencia
        private void lblTexto_Click(object sender, EventArgs e)
        {

        }
    }
}
```

PANTALLA DE CARGA
Ubicación: Controlador

Descripción:
Controlador encargado de gestionar el inicio de sesión.

Responsabilidad:

Recibir datos del formulario
Validar credenciales
Coordinar con el modelo

Métodos principales:

Login(string usuario, string pass, FrmLogin vista)
Procesa el inicio de sesión y decide si el acceso es válido.

Importancia:
Separa la lógica del login de la interfaz gráfica.

apli | apli.FrmLogin

```
using System;
using System.Drawing;
using System.Windows.Forms;
using apli.Controlador;

namespace apli
{
    4 referencias
    public partial class FrmLogin : Form
    {
        private Button btnRegistro;

        1 referencia
        public FrmLogin()
        {
            InitializeComponent();

            // ◇ Botón Registrarse (bien posicionado)
            btnRegistro = new Button();
            btnRegistro.Text = "Registrarse";
            btnRegistro.Size = new Size(120, 30);

            // 📍 POSICIÓN SEGURA (debajo del botón login)
            btnRegistro.Location = new Point(90, 185);

            // 🎨 Para que se vea claramente
            btnRegistro.BackColor = Color.Gray;
            btnRegistro.ForeColor = Color.White;
            btnRegistro.FlatStyle = FlatStyle.Flat;

            btnRegistro.Click += btnRegistro_Click;

            this.Controls.Add(btnRegistro);
        }

        1 referencia
        private void btnLogin_Click(object sender, EventArgs e)
        {
            new LoginController().Login(
                txtUsuario.Text,
                txtContrasena.Text,
                this
            );
        }

        1 referencia
        private void btnRegistro_Click(object sender, EventArgs e)
        {
            FrmRegistro registro = new FrmRegistro();
            registro.ShowDialog();
        }

        1 referencia
        public void SetEstado(string msg)
        {
            lblEstado.Text = msg;
        }

        1 referencia
        private void FrmLogin_Load(object sender, EventArgs e)
        {

        }
    }
}
```

0 % | 0 | 1

REGISTRO
Ubicación: Vista

Descripción:
Formulario para registrar nuevos usuarios.

Responsabilidad:

Capturar datos personales
Enviar información al controlador

Eventos principales:

btnGuardar_Click
Ejecuta el registro del usuario.

Importancia:
Permite crear nuevas cuentas dentro del sistema.

```
namespace apli
{

    partial class FrmRegistro
    {
        private System.ComponentModel.IContainer components = null;

        private TextBox txtNombre;
        private TextBox txtApellido;
        private TextBox txtUsuario;
        private TextBox txtContrasena;
        private Button btnGuardar;
        private Label lblTitulo;


        protected override void Dispose(bool disposing)
        {
            if (disposing && (components != null))
                components.Dispose();
            base.Dispose(disposing);
        }

        private void InitializeComponent()
        {
            txtNombre = new TextBox();
            txtApellido = new TextBox();
            txtUsuario = new TextBox();
            txtContrasena = new TextBox();
            btnGuardar = new Button();
            lblTitulo = new Label();

            SuspendLayout();

            // Título
            lblTitulo.Text = "Registro";
            lblTitulo.Font = new Font("Segoe UI", 16, FontStyle.Bold);
            lblTitulo.Location = new Point(80, 20);
            lblTitulo.Size = new Size(150, 30);

            // Campos
            txtNombre.PlaceholderText = "Nombre";
            txtApellido.PlaceholderText = "Apellido";
            txtUsuario.PlaceholderText = "Usuario";
            txtContrasena.PlaceholderText = "Contraseña";
            txtContrasena.PasswordChar = '•';

            txtNombre.Location = new Point(60, 70);
            txtApellido.Location = new Point(60, 110);
            txtUsuario.Location = new Point(60, 150);
            txtContrasena.Location = new Point(60, 190);

            txtNombre.Size = txtApellido.Size =
            txtUsuario.Size = txtContrasena.Size = new Size(200, 23);

            // Botón
            btnGuardar.Text = "Guardar";
            btnGuardar.Location = new Point(100, 230);
            btnGuardar.Size = new Size(120, 30);
            btnGuardar.BackColor = Color.FromArgb(30, 86, 49);
            btnGuardar.ForeColor = Color.White;
            btnGuardar.FlatStyle = FlatStyle.Flat;
            btnGuardar.Click += btnGuardar_Click;
```

No se encontraron problemas.

CONTROLADOR
Ubicación: Controlador

Descripción:
Controlador encargado de gestionar el inicio de sesión.

Responsabilidad:

Recibir datos del formulario
Validar credenciales
Coordinar con el modelo

Métodos principales:

Login(string usuario, string pass, FrmLogin vista)

```
using apli.Modelo;

namespace apli.Controlador
{
    1 referencia
    public class LoginController
    {
        UsuarioDAO dao = new();

        1 referencia
        public void Login(string u, string p, FrmLogin f)
        {
            var user = dao.Login(u, p);

            if (user != null)
            {
                new FrmBienvenida(user).Show();
                f.Hide();
            }
            else
            {
                f.SetEstado("Datos incorrectos");
            }
        }
    }
}
```

CONTRASEÑA
Ubicación: Modelo

Descripción:
Clase utilitaria para validar contraseñas.

Responsabilidad:

Evaluar la seguridad de una contraseña
Validar requisitos mínimos

Métodos principales:

CalcularFortaleza(string pass)
Retorna un nivel de seguridad de la contraseña.
EsValida(string pass)
Verifica si cumple con los requisitos mínimos.

FrmLogin.cs [Diseño] | ConexionDB.cs | FrmLogin.Designer.cs

apli | apli.Mod

```
using System.Text.RegularExpressions;

namespace apli.Modelo
{
    public static class PasswordUtils
    {
        public static int CalcularFortaleza(string pass)
        {
            if (string.IsNullOrEmpty(pass)) return 0;

            int score = 0;
            if (pass.Length >= 8) score++;
            if (Regex.IsMatch(pass, "[A-Z]")) score++;
            if (Regex.IsMatch(pass, "[a-z]")) score++;
            if (Regex.IsMatch(pass, "[0-9]")) score++;
            if (Regex.IsMatch(pass, "[!@#$%^&*]")) score++;

            return Math.Min(4, score);
        }

        public static bool EsValida(string pass)
        {
            return pass.Length >= 8 &&
                   Regex.IsMatch(pass, "[A-Z]") &&
                   Regex.IsMatch(pass, "[a-z]") &&
                   Regex.IsMatch(pass, "[0-9]") &&
                   Regex.IsMatch(pass, "[!@#$%^&*]");
        }
    }
}
```

PROGRAM
Ubicación: Carpeta raíz del proyecto

Descripción:
Es la clase principal del sistema. Representa el punto de entrada de la aplicación.

Responsabilidad:

Inicializar la aplicación Windows Forms
Verificar la conexión a la base de datos
Ejecutar el formulario principal (Login)

Métodos principales:

Main()
Inicia la aplicación y valida la conexión con la base de datos antes de mostrar la interfaz.

Importancia:
Garantiza que la aplicación solo se ejecute si existe conexión con MySQL, evitando errores posteriores.

Debug | Any CPU

nexionDB.cs | FrmLogin.Designer.cs | FrmRegistro.cs

apli

```
using apli.Modelo;

namespace apli
{

    internal static class Program
    {
        [STAThread]
        0 referencias
        static void Main()
        {
            ApplicationConfiguration.Initialize();

            if (!ConexionDB.ProbarConexion())
            {
                MessageBox.Show("Error MySQL");
                return;
            }

            Application.Run(new FrmLogin());
        }
    }
}
```

MODELO

FrmLogin.Designer.cs | FrmRegistro.cs [Diseño] | FrmRegistro.Designer.cs | Log

apli | apli.Controlador.RegistroCo

```
using apli.Modelo;

namespace apli.Controlador
{

    public class RegistroController
    {
        UsuarioDAO dao = new();


        public void Registrar(string n, string a, string e, string u, string p)
        {
            if (!PasswordUtils.EsValida(p))
            {
                MessageBox.Show("Contraseña débil");
                return;
            }

            dao.Insertar(new Usuario
            {
                Nombre = n,
                Apellido = a,
                Email = e,
                NombreUsuario = u,
                Contrasena = p
            });

            MessageBox.Show("Registrado");
        }
    }
}
```

USUARIO

Archivo Editar Ver Git Proyecto Compilar Depurar Prueba Analizar

Debug | Any CPU | apli

Cuadro de herramientas | Orígenes de datos

FrmRegistro.cs [Diseño] | FrmRegistro.Designer.cs | LoginController.cs

apli | apli.Modelo.Usua

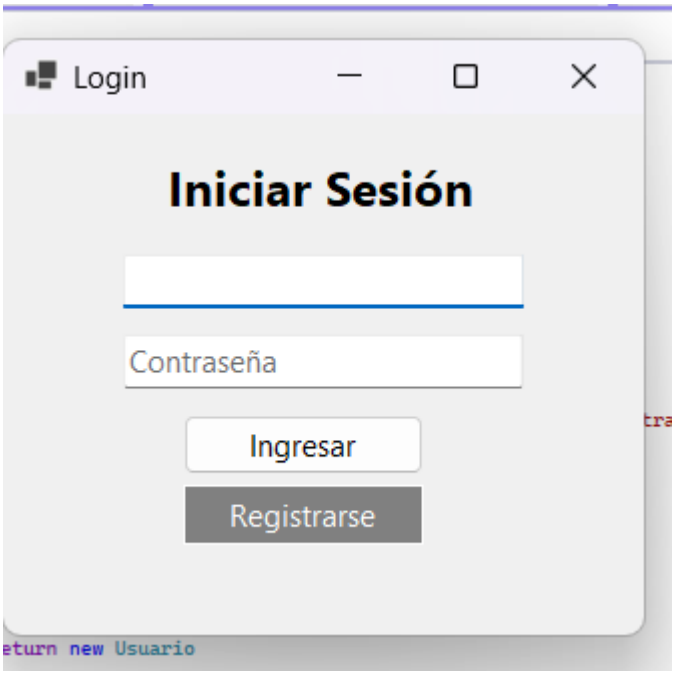
```
namespace apli.Modelo
{

    public class Usuario
    {

        public int Id { get; set; }

        public string Nombre { get; set; } = "";

        public string Apellido { get; set; } = "";

        public string Email { get; set; } = "";

        public string NombreUsuario { get; set; } = "";

        public string Contrasena { get; set; } = "";

        public byte[]? Foto { get; set; }

        public int NivelSeguridad { get; set; }

        public bool Bloqueado { get; set; }

        public int Intentos { get; set; }

        public DateTime? UltimoAcceso { get; set; }

        public bool Activo { get; set; } = true;

        1 referencia
        public string NombreCompleto => $"{Nombre} {Apellido}";
    }
}
```

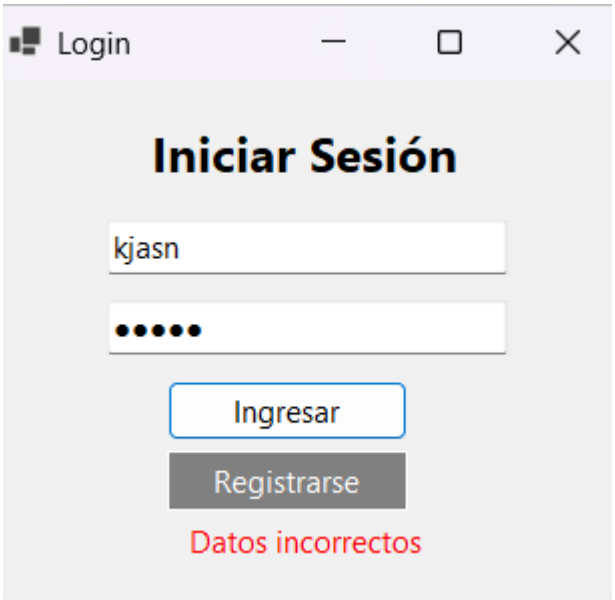
CONEXION CON MYSQL

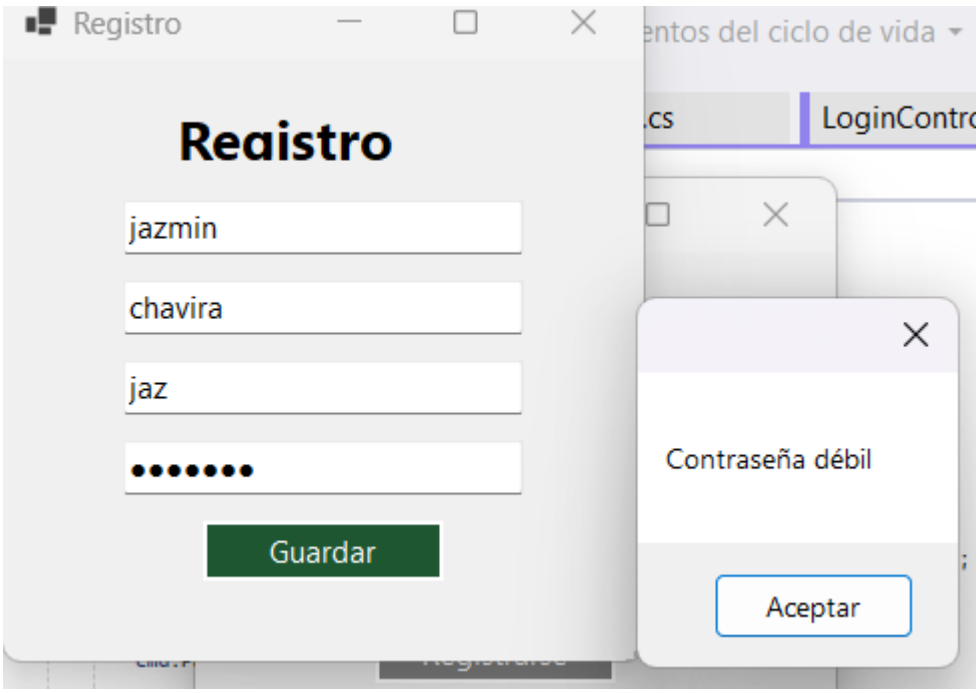
FrmRegistro.cs [Diseño] | FrmRegistro.Designer.cs | LoginControlle

apli | apli.M

```
        private MySqlConnection Conn => ConexionDB.Instancia.GetConexion();


        public bool Insertar(Usuario u)
        {
            string sql = @"INSERT INTO usuarios
            (nombre, apellido, email, usuario, contrasena)
            VALUES (@n,@a,@e,@u,@c)";

            using var cmd = new MySqlCommand(sql, Conn);
            cmd.Parameters.AddWithValue("@n", u.Nombre);
            cmd.Parameters.AddWithValue("@a", u.Apellido);
            cmd.Parameters.AddWithValue("@e", u.Email);
            cmd.Parameters.AddWithValue("@u", u.NombreUsuario);
            cmd.Parameters.AddWithValue("@c", u.Contrasena);

            return cmd.ExecuteNonQuery() > 0;
        }


        public Usuario? Login(string usuario, string pass)
        {
            string sql = "SELECT * FROM usuarios WHERE usuario=@u AND contrasena=@p";

            using var cmd = new MySqlCommand(sql, Conn);
            cmd.Parameters.AddWithValue("@u", usuario);
            cmd.Parameters.AddWithValue("@p", pass);

            using var r = cmd.ExecuteReader();
            if (r.Read())
            {
                return new Usuario
                {
                    Nombre = r.GetString("nombre"),
                    Apellido = r.GetString("apellido"),
                    NombreUsuario = r.GetString("usuario")
                };
            }
            return null;
        }


        public bool UsuarioExiste(string user)
        {
            using var cmd = new MySqlCommand(
                "SELECT COUNT(*) FROM usuarios WHERE usuario=@u", Conn);
            cmd.Parameters.AddWithValue("@u", user);
            return Convert.ToInt32(cmd.ExecuteScalar()) > 0;
        }
    }
}
```


Login
Iniciar Sesión
Contraseña
Ingresar
Registrarse
eturn new Usuario


Login
Iniciar Sesión
kjasn
Ingresar
Registrarse
Datos incorrectos


Registro
Registro
jazmin
chavira
jaz
Guardar
entos del ciclo de vida
.cs
LoginContro
Contraseña débil
Aceptar